Stream-API

Carsten Gips (HSBI)

TL;DR
Mit der Collection-API existiert in Java die Möglichkeit, Daten auf verschiedenste Weisen zu speichern (`Collection<T>`). Mit der Stream-API gibt es die Möglichkeit, diese Daten in einer Art Pipeline zu verarbeiten. Ein `Stream<T>` ist eine Folge von Objekten vom Typ `T`. Die Verarbeitung der Daten ist "lazy", d.h. sie erfolgt erst auf Anforderung (durch die terminale Operation).
Ein Stream hat eine Datenquelle und kann beispielsweise über `Collection#stream()` oder `Stream.of()` angelegt werden. Streams speichern keine Daten. Die Daten werden aus der verbundenen Datenquelle geholt.
Auf einem Stream kann man eine Folge von intermediären Operationen wie `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` … durchführen. Alle diese Operationen arbeiten auf dem Stream und erzeugen einen neuen Stream als Ergebnis. Dadurch kann die typische Pipeline-artige Verkettung der Operationen ermöglicht werden. Die intermediären Operationen werden erst ausgeführt, wenn der Stream durch eine terminale Operation geschlossen wird.
Terminale Operationen wie `count()`, `forEach()`, `allMatch()` oder `collect()`

- `collect(Collectors.toList())` (bzw. direkt mit `stream.toList()` (ab Java16))
- `collect(Collectors.toSet())`
- `collect(Collectors.toCollection(LinkedList::new))` (als `Supplier<T>`)

stoßen die Verarbeitung des Streams an und schließen den Stream damit ab.
Wir können hier nur die absoluten Grundlagen betrachten. Die Stream-API ist sehr groß und mächtig und lohnt die weitere selbstständige Auseinandersetzung :-)

Videos

- VL Stream-API
- Demo Stream-API
- Demo Vordefinierte funktionale Interfaces im JDK

Unterlagen

- PDF-Version

---

1 Motivation

Es wurden Studis, Studiengänge und Fachbereiche modelliert (aus Gründen der Übersichtlichkeit einfach als Record-Klassen).

Nun soll pro Fachbereich die Anzahl der Studis ermittelt werden, die bereits 100 ECTS oder mehr haben. Dazu könnte man über alle Studiengänge im Fachbereich iterieren, und in der inneren Schleife über alle Studis im Studiengang. Dann filtert man alle Studis, deren ECTS größer 100 sind und erhöht jeweils den Zähler:

```
1  public record Studi(String name, int credits) {}
2  public record Studiengang(String name, List<Studi> studis) {}
3  public record Fachbereich(String name, List<Studiengang> studiengaenge) {}
4
5  private static long getCountFB(Fachbereich fb) {
6      long count = 0;
7      for (Studiengang sg : fb.studiengaenge()) {
```

```
        for (Studi s : sg.studis()) {
            if (s.credits() > 100) count += 1;
        }
    }
    return count;
}
```

Dies ist ein Beispiel, welches klassisch in OO-Manier als Iteration über Klassen realisiert ist. (Inhaltlich ist es vermutlich nicht sooo sinnvoll.)

2 Innere Schleife mit Streams umgeschrieben

```
private static long getCountSG(Studiengang sg) {
    return sg.studis().stream()
                      .map(Studi::credits)
                      .filter(c -> c > 100)
                      .count();
}

private static long getCountFB2(Fachbereich fb) {
    long count = 0;
    for (Studiengang sg : fb.studiengaenge()) {
        count += getCountSG(sg);
    }
    return count;
}
```

2.1 Erklärung des Beispiels

Im Beispiel wurde die innere Schleife in einen Stream ausgelagert.

Mit der Methode `Collection#stream()` wird aus der Collection ein neuer Stream erzeugt. Auf diesem wird für jedes Element durch die Methode `map()` die Methode `Studi#credits()` angewendet, was aus einem Strom von `Studi` einen Strom von `Integer` macht. Mit `filter()` wird auf jedes Element das Prädikat `c -> c > 100` angewendet und alle Elemente aus dem Strom entfernt, die der Bedingung nicht entsprechen. Am Ende wird mit `count()` gezählt, wie viele Elemente im Strom enthalten sind.

2.2 Was ist ein Stream?

Ein “Stream” ist ein Strom (Folge) von Daten oder Objekten. In Java wird die Collections-API für die Speicherung von Daten (Objekten) verwendet. Die Stream-API dient zur Iteration über diese Daten und entsprechend zur Verarbeitung der Daten. In Java speichert ein Stream keine Daten.

Das Konzept kommt aus der funktionalen Programmierung und wurde in Java nachträglich eingebaut (wobei dieser Prozess noch lange nicht abgeschlossen zu sein scheint).

In der funktionalen Programmierung kennt man die Konzepte “map”, “filter” und “reduce”: Die Funktion “map()” erhält als Parameter eine Funktion und wendet diese auf alle Elemente eines Streams an. Die Funktion “filter()”

bekommt ein Prädikat als Parameter und prüft jedes Element im Stream, ob es dem Prädikat genügt (also ob das Prädikat mit dem jeweiligen Element zu `true` evaluiert - die anderen Objekte werden entfernt). Mit "reduce()" kann man Streams zu einem einzigen Wert zusammenfassen (denken Sie etwa an das Aufsummieren aller Elemente eines Integer-Streams). Zusätzlich kann man in der funktionalen Programmierung ohne Probleme unendliche Ströme darstellen: Die Auswertung erfolgt nur bei Bedarf und auch dann auch nur so weit wie nötig. Dies nennt man auch "*lazy evaluation*".

Die Streams in Java versuchen, diese Konzepte aus der funktionalen Programmierung in die objektorientierte Programmierung zu übertragen. Ein Stream in Java hat eine Datenquelle, von wo die Daten gezogen werden - ein Stream speichert selbst keine Daten. Es gibt "intermediäre Operationen" auf einem Stream, die die Elemente verarbeiten und das Ergebnis als Stream zurückliefern. Daraus ergibt sich typische Pipeline-artige Verkettung der Operationen. Allerdings werden diese Operationen erst durchgeführt, wenn eine "terminale Operation" den Stream "abschließt". Ein Stream ohne eine terminale Operation macht also tatsächlich *nichts*.

Die Operationen auf dem Stream sind üblicherweise zustandslos, können aber durchaus auch einen Zustand haben. Dies verhindert üblicherweise die parallele Verarbeitung der Streams. Operationen sollten aber nach Möglichkeit keine *Seiteneffekte* haben, d.h. keine Daten außerhalb des Streams modifizieren. Operationen dürfen auf keinen Fall die Datenquelle des Streams modifizieren!

3 Erzeugen von Streams

```
List<String> l1 = List.of("Hello", "World", "foo", "bar", "wuppie");
Stream<String> s1 = l1.stream();

Stream<String> s2 = Stream.of("Hello", "World", "foo", "bar", "wuppie");

Random random = new Random();
Stream<Integer> s3 = Stream.generate(random::nextInt);

Pattern pattern = Pattern.compile(" ");
Stream<String> s4 = pattern.splitAsStream("Hello world! foo bar wuppie!");
```

Dies sind möglicherweise die wichtigsten Möglichkeiten, in Java einen Stream zu erzeugen.

Ausgehend von einer Klasse aus der Collection-API kann man die Methode `Collection#stream()` aufrufen und bekommt einen seriellen Stream.

Alternativ bietet das Interface `Stream` verschiedene statische Methoden wie `Stream.of()` an, mit deren Hilfe Streams angelegt werden können. Dies funktioniert auch mit Arrays ...

Und schließlich kann man per `Stream.generate()` einen Stream anlegen, wobei als Argument ein "Supplier" (Interface `java.util.function.Supplier<T>`) übergeben werden muss. Dieses Argument wird dann benutzt, um die Daten für den Stream zu generieren.

Wenn man aufmerksam hinschaut, findet man an verschiedensten Stellen die Möglichkeit, die Daten per Stream zu verarbeiten, u.a. bei regulären Ausdrücken.

Man kann per `Collection#parallelStream()` auch parallele Streams erzeugen, die intern das "Fork&Join-Framework" nutzen. Allerdings sollte man nur dann parallele Streams anlegen, wenn dadurch

tatsächlich Vorteile durch die Parallelisierung zu erwarten sind (Overhead!).

4 Intermediäre Operationen auf Streams

```
private static void dummy(Studiengang sg) {
    sg.studis().stream()
            .peek(s -> System.out.println("Looking at: " + s.name()))
            .map(Studi::credits)
            .peek(c -> System.out.println("This one has: " + c + " ECTS"))
            .filter(c -> c > 5)
            .peek(c -> System.out.println("Filtered: " + c))
            .sorted()
            .forEach(System.out::println);
}
```

An diesem (weitestgehend sinnfreien) Beispiel werden einige intermediäre Operationen demonstriert.

Die Methode `peek()` liefert einen Stream zurück, die aus den Elementen des Eingabestroms bestehen. Auf jedes Element wird die Methode `void accept(T)` des `Consumer<T>` angewendet (Argument der Methode), was aber nicht zu einer Änderung der Daten führt. **Hinweis**: Diese Methode dient vor allem zu Debug-Zwecken! Durch den Seiteneffekt kann die Methode eine schlechtere Laufzeit zur Folge haben oder sogar eine sonst mögliche parallele Verarbeitung verhindern oder durch eine parallele Verarbeitung verwirrende Ergebnisse zeigen!

Die Methode `map()` liefert ebenfalls einen Stream zurück, der durch die Anwendung der Methode `R apply(T)` der als Argument übergebenen `Function<T,R>` auf jedes Element des Eingabestroms entsteht. Damit lassen sich die Elemente des ursprünglichen Streams verändern; für jedes Element gibt es im Ergebnis-Stream ebenfalls ein Element (der Typ ändert sich, aber nicht die Anzahl der Elemente).

Mit der Methode `filter()` wird ein Stream erzeugt, der alle Objekte des Eingabe-Streams enthält, auf denen die Anwendung der Methode `boolean test(T)` des Arguments `Predicate<T>` zu `true` evaluiert (der Typ und Inhalt der Elemente ändert sich nicht, aber die Anzahl der Elemente).

Mit `sorted()` wird ein Stream erzeugt, der die Elemente des Eingabe-Streams sortiert (existiert auch mit einem `Comparator<T>` als Parameter).

Diese Methoden sind alles **intermediäre** Operationen. Diese arbeiten auf einem Stream und erzeugen einen neuen Stream und werden erst dann ausgeführt, wenn eine terminale Operation den Stream abschließt.

Dabei sind die gezeigten intermediären Methoden bis auf `sorted()` ohne inneren Zustand. `sorted()` ist eine Operation mit innerem Zustand (wird für das Sortieren benötigt). Dies kann ordentlich in Speicher und Zeit zuschlagen und u.U. nicht/nur schlecht parallelisierbar sein. Betrachten Sie den fiktiven parallelen Stream `stream.parallel().sorted().skip(42)`: Hier müssen erst *alle* Elemente sortiert werden, bevor mit `skip(42)` die ersten 42 Elemente entfernt werden. Dies kann auch nicht mehr parallel durchgeführt werden.

Die Methode `forEach()` schließlich ist eine **terminale** Operation, die auf jedes Element des Eingabe-Streams die Methode `void accept(T)` des übergebenen `Consumer<T>` anwendet. Diese Methode ist eine **terminale Operation**, d.h. sie führt zur Auswertung der anderen *intermediären* Operationen und schließt den Stream ab.

5 Was tun, wenn eine Methode Streams zurückliefert

Wir konnten vorhin nur die innere Schleife in eine Stream-basierte Verarbeitung umbauen. Das Problem ist: Die äußere Schleife würde einen Stream liefern (Stream von Studiengängen), auf dem wir die `map`-Funktion anwenden müssten und darin dann für jeden Studiengang einen (inneren) Stream mit den Studis eines Studiengangs verarbeiten müssten.

```
private static long getCountSG(Studiengang sg) {
    return sg.studis().stream().map(Studi::credits).filter(c -> c > 100).count
        ();
}

private static long getCountFB2(Fachbereich fb) {
    long count = 0;
    for (Studiengang sg : fb.studiengaenge()) {
        count += getCountSG(sg);
    }
    return count;
}
```

Dafür ist die Methode `flatMap()` die Lösung. Diese Methode bekommt als Argument ein Objekt vom Typ `Function<? super T, ? extends Stream<? extends R>>` mit einer Methode `Stream<? extends R> apply(T)`. Die Methode `flatMap()` verarbeitet den Stream in zwei Schritten:

1. Mappe über alle Elemente des Eingabe-Streams mit der Funktion. Im Beispiel würde also aus einem `Stream<Studiengang>` jeweils ein `Stream<Stream<Studi>>`, also alle `Studiengang`-Objekte werden durch je ein `Stream<Studi>`-Objekt ersetzt. Wir haben jetzt also einen Stream von `Stream<Studi>`-Objekten, also einen `Stream<Stream<Studi>>`.
2. "Klopfe den verschachtelten Stream wieder flach", d.h. nimm die einzelnen `Studi`-Objekte aus den `Stream<Studi>`-Objekten und setze diese stattdessen in den Stream. Das Ergebnis ist dann wie gewünscht ein `Stream<Studi>` (Stream mit `Studi`-Objekten).

```
private static long getCountFB3(Fachbereich fb) {
    return fb.studiengaenge().stream()
            .flatMap(sg -> sg.studis().stream())
            .map(Studi::credits)
            .filter(c -> c > 100)
            .count();
}
```

Zum direkten Vergleich hier noch einmal der ursprüngliche Code mit zwei verschachtelten Schleifen und entsprechenden Hilfsvariablen:

```
private static long getCountFB(Fachbereich fb) {
    long count = 0;
    for (Studiengang sg : fb.studiengaenge()) {
        for (Studi s : sg.studis()) {
            if (s.credits() > 100) count += 1;
        }
    }
    return count;
```

```
}
```

Während `map` also eine Funktion $f : T \mapsto R$ auf alle Elemente des Streams anwendet und so aus einem `stream<T>` einen `stream<R>` erzeugt, wendet `flatMap` eine Funktion $f : T \mapsto$ stream $<R>$ auf alle Elemente des Streams an und packt die Ergebnis-Streams `stream<R>` wieder aus, weshalb man im Ergebnis wie bei `map` aus einem `stream<T>` einen `stream<R>` erhält.

6 Streams abschließen: Terminale Operationen

```
Stream<String> s = Stream.of("Hello", "World", "foo", "bar", "wuppie");

long count = s.count();
s.forEach(System.out::println);
String first = s.findFirst().get();
Boolean b = s.anyMatch(e -> e.length() > 3);

List<String> s1 = s.collect(Collectors.toList());
List<String> s2 = s.toList();   // ab Java16
Set<String> s3 = s.collect(Collectors.toSet());
List<String> s4 = s.collect(Collectors.toCollection(LinkedList::new));
```

Streams müssen mit ***einer* terminalen Operation** abgeschlossen werden, damit die Verarbeitung tatsächlich angestoßen wird (*lazy evaluation*).[1]

Es gibt viele verschiedene terminale Operationen. Wir haben bereits `count()` und `forEach()` gesehen. In der Sitzung zu "Optionals" werden wir noch `findFirst()` näher kennenlernen.

Daneben gibt es beispielsweise noch `allMatch()`, `anyMatch()` und `noneMatch()`, die jeweils ein Prädikat testen und einen Boolean zurückliefern (matchen alle, mind. eines oder keines der Objekte im Stream).

Mit `min()` und `max()` kann man sich das kleinste und das größte Element des Streams liefern lassen. Beide Methoden benötigen dazu einen `Comparator<T>` als Parameter.

Mit der Methode `collect()` kann man eine der drei Methoden aus `Collectors` über den Stream laufen lassen und eine `Collection` erzeugen lassen:

1. `toList()` sammelt die Elemente in ein `List`-Objekt (bzw. direkt mit `stream.toList()` (ab Java16))
2. `toSet()` sammelt die Elemente in ein `Set`-Objekt
3. `toCollection()` sammelt die Elemente durch Anwendung der Methode `T get()` des übergebenen `Supplier<T>`-Objekts auf

Die ist nur die sprichwörtliche "Spitze des Eisbergs"! Es gibt viele weitere Möglichkeiten, sowohl bei den intermediären als auch den terminalen Operationen. Schauen Sie in die Dokumentation!

Demo: streams.Demo

[1] Anmerkung: Das obige Beispiel dient als Überblick gebräuchlicher terminaler Operationen, es ist nicht als lauffähiges Programm gedacht! Auf einem Stream kann immer nur **eine** terminale Operation ausgeführt werden - d.h. nach der Ausführung von `s.count()` wäre der Stream `s` verarbeitet und es können keine weiteren Operationen auf diesem Stream durchgeführt werden. Dito für die anderen gezeigten terminalen Operationen.

7 Spielregeln

- Operationen dürfen nicht die Stream-Quelle modifizieren
- Operationen können die Werte im Stream ändern (`map`) oder die Anzahl (`filter`)
- Keine Streams in Attributen/Variablen speichern oder als Argumente übergeben: Sie könnten bereits "gebraucht" sein!

  => Ein Stream sollte immer sofort nach der Erzeugung benutzt werden
- Operationen auf einem Stream sollten keine Seiteneffekte (Veränderungen von Variablen/Attributen außerhalb des Streams) haben (dies verhindert u.U. die parallele Verarbeitung)

8 Wrap-Up

`Stream<T>`: Folge von Objekten vom Typ `T`, Verarbeitung "lazy" (Gegenstück zu `Collection<T>`: Dort werden Daten **gespeichert**, hier werden Daten **verarbeitet**)

Fließband-Metapher
Einen Stream kann man sich vielleicht wie ein Fließband in einer Fabrik vorstellen: Die Daten werden auf dem Fließband in eine Richtung transportiert und durchlaufen verschiedene Stationen, wo auf den Daten gearbeitet wird. In manchen Stationen werden Objekte vom Fließband geschubst (Daten herausgefiltert), in manchen Stationen werden die Objekte bearbeitet (Daten verändert), in manchen Stationen werden aus mehreren Teilen neue Objekte gebaut …
Es ist nur eine Metapher! Sie endet spätestens damit, dass die Streams *lazy* sind und dass sämtliche Operationen erst dann ausgeführt werden, wenn eine terminale Operation den Stream abschließt.

- Neuen Stream anlegen: `Collection#stream()` oder `Stream.of()` …
- Intermediäre Operationen: `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` …
- Terminale Operationen: `count()`, `forEach()`, `allMatch()`, `collect()` …
  - `collect(Collectors.toList())`
  - `collect(Collectors.toSet())`
  - `collect(Collectors.toCollection())` (mit `Supplier<T>`)

- Streams speichern keine Daten
- Intermediäre Operationen laufen erst bei Abschluss des Streams los
- Terminale Operation führt zur Verarbeitung und Abschluss des Streams

Schöne Doku: "The Stream API", und auch "Package java.util.stream".

9 Zum Nachlesen

- Oracle Corporation (2025)
- Ullenboom (2021, Kap. 17.3–17.6)

Lernziele

- k2: Ich verstehe, dass Streams die Daten nicht sofort verarbeiten ('lazy' Verarbeitung)
- k2: Ich verstehe, dass ich mit map() den Typ (und Inhalt) von Objekten im Stream, aber nicht die Anzahl verändere
- k2: Ich verstehe, dass ich mit filter() die Anzahl der Objekte im Stream, aber nicht deren Typ (und Inhalt) verändere
- k2: Ich verstehe, warum Streams nicht in Attributen gehalten oder als Parameter herumgereicht werden sollten
- k3: Ich kann einen Stream erzeugen
- k3: Ich kann verschiedene intermediäre Operationen verketten
- k3: Ich kann mit einer terminalen Operation einen Stream abschließen und damit die Berechnung durchführen
- k3: Ich kann flatMap() einsetzen

Challenges

Betrachten Sie den folgenden Java-Code:

```java
record Cat(int weight){};

public class Main {
    public static void main(String... args) {
        List<Cat> clouder = new ArrayList<>();
        clouder.add(new Cat(100));  clouder.add(new Cat(1));  clouder.add(new Cat(10));

        sumOverWeight(8, clouder);
    }

    private static int sumOverWeight(int threshold, List<Cat> cats) {
        int result = 0;
        for (Cat c : cats) {
            int weight = c.weight();
            if (weight > threshold) {
                result += weight;
            }
        }
        return result;
    }
}
```

Schreiben Sie die Methode `sumOverWeight` unter Beibehaltung der Funktionalität so um, dass statt der `for`-Schleife und der `if`-Abfrage Streams und Stream-Operationen eingesetzt werden. Nutzen Sie passende Lambda-Ausdrücke und nach Möglichkeit Methodenreferenzen.

Betrachten Sie den folgenden Java-Code:

```java
public class Main {
    public static String getParameterNamesJson(String[] parameterNames) {
        if (parameterNames.length == 0) {
            return "[]";
        }

        StringBuilder sb = new StringBuilder();
        sb.append("[");
        for (int i = 0; i < parameterNames.length; i++) {
            if (i > 0) {
                sb.append(", ");
            }
            sb.append("\"").append(escapeJson(parameterNames[i])).append("\"");
        }
        sb.append("]");
        return sb.toString();
    }

    private static String escapeJson(String parameterName) {
        // does something or another ...
    }
}
```

Schreiben Sie die Methode `getParameterNamesJson` unter Beibehaltung der Funktionalität so um, dass statt der `for`-Schleife und der `if`-Abfrage Streams und Stream-Operationen eingesetzt werden. Nutzen Sie passende Lambda-Ausdrücke und nach Möglichkeit auch Methodenreferenzen.

Quellen

Oracle Corporation. 2025. „Learn Java“. 2025. https://dev.java/learn/.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** d801da0 (lecture: add 'assembly line' metaphor (Stream API), 2025-05-26)